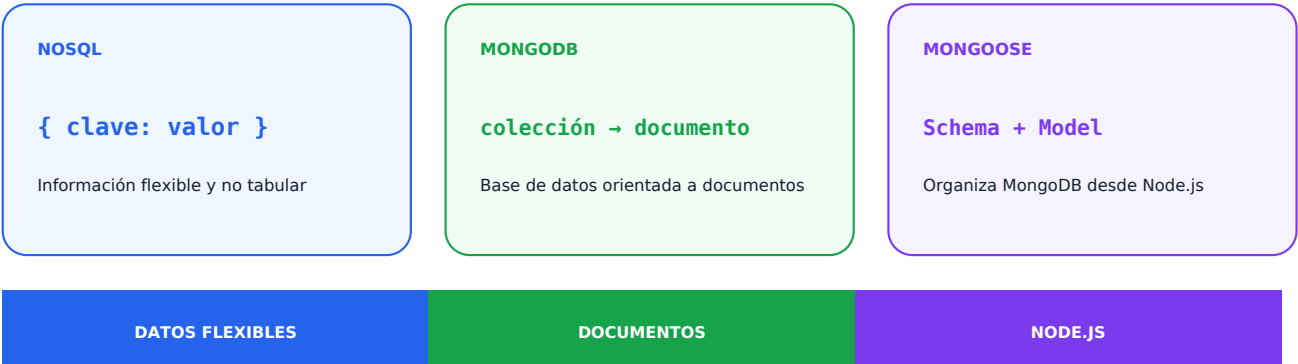




MongoDB y Mongoose

Bases de datos NoSQL, documentos, modelos y consultas



ESTUDIANTE	BOOTCAMP	FECHA
Juan Andrés Suárez Gaviria	Desarrollo web fullstack	9 de septiembre de 2026

Actividad de investigación | Persistencia de datos con MongoDB

Bases de datos no relacionales

DEFINICIÓN

Una base de datos no relacional, también llamada NoSQL, guarda la información con modelos distintos a las tablas y relaciones rígidas de una base SQL. Puede usar documentos, pares clave-valor, columnas o grafos.

Características principales

FLEXIBILIDAD

Los registros pueden tener campos diferentes.

ESCALABILIDAD

Facilita distribuir grandes volúmenes de datos.

VELOCIDAD

Optimiza consultas según el modelo utilizado.

Tipos frecuentes de bases NoSQL

Tipo	Forma de guardar datos	Ejemplo de uso
Documentos	Objetos parecidos a JSON	Catálogos, usuarios y contenido
Clave-valor	Una clave identifica cada valor	Sesiones, caché y configuraciones
Columnas	Datos agrupados por familias de columnas	Analítica y grandes volúmenes
Grafos	Nodos conectados mediante relaciones	Redes sociales y recomendaciones

SQL y NoSQL: diferencia general

Aspecto	Base relacional (SQL)	Base no relacional (NoSQL)
Estructura	Tablas con filas y columnas	Documentos, claves, columnas o grafos
Esquema	Definido antes de guardar	Flexible según la tecnología
Relaciones	Llaves y JOIN	Datos embebidos o referencias
Ideal para	Relaciones y reglas muy estructuradas	Datos variables y crecimiento horizontal

NoSQL no significa “sin estructura”. Significa que la información no depende únicamente del modelo relacional y puede adaptarse con mayor libertad.

¿Qué es MongoDB?

DEFINICIÓN

MongoDB es una base de datos NoSQL orientada a documentos. Guarda la información en documentos similares a JSON y los agrupa en colecciones. Internamente utiliza BSON, un formato binario que admite más tipos de datos.

Cómo organiza la información

1 Base de datos

Ejemplo: tienda

2 Colección

Ejemplo: productos

3 Documento

Un producto completo

4 Campo

nombre, precio, stock

Ejemplo de documento

DOCUMENTO BSON / JSON

```
{
  "_id": "66d5...",
  "nombre": "Teclado mecánico",
  "precio": 180000,
  "disponible": true,
  "categorias": ["tecnología", "oficina"],
  "fabricante": {
    "nombre": "KeyLab",
    "pais": "Colombia"
  }
}
```

Concepto	Función
_id	Identificador único
Colección	Agrupar documentos
Documento	Representa un registro
Campo	Guarda un dato

Operaciones básicas en MongoDB

MONGOSH

```
// Crear un documento
db.productos.insertOne({ nombre: "Mouse", precio: 65000 })

// Consultar, actualizar y eliminar
db.productos.find({ precio: { $lt: 100000 } })
db.productos.updateOne({ nombre: "Mouse" }, { $set: { precio: 60000 } })
db.productos.deleteOne({ nombre: "Mouse" })
```

MongoDB permite documentos flexibles, pero una aplicación puede agregar validaciones para controlar campos, tipos y valores aceptados.

¿Qué es Mongoose?

DEFINICIÓN

Mongoose es una librería ODM para Node.js y MongoDB. Permite definir esquemas, crear modelos, validar datos y realizar consultas con JavaScript, manteniendo el código del backend más organizado.

Lugar de Mongoose en el backend

1 Cliente

Envía una petición HTTP

2 Express

Recibe la ruta

3 Mongoose

Valida y consulta

4 MongoDB

Guarda documentos

Conceptos principales

Concepto	Qué representa	Ejemplo
Schema	Reglas y forma de los datos	nombre: String
Model	Objeto para consultar una colección	Producto.find()
Document	Instancia concreta de un modelo	Un producto guardado
Validation	Comprueba los valores recibidos	required, min, enum

Instalación y conexión

TERMINAL

```
npm init -y
npm install mongoose dotenv
```

CONFIG/DATABASE.JS

```
require("dotenv").config();
const mongoose = require("mongoose");

async function conectarDB() {
  await mongoose.connect(process.env.MONGODB_URI);
  console.log("MongoDB conectado");
}

module.exports = conectarDB;
```

.ENV

```
MONGODB_URI=mongodb://127.0.0.1:27017/tienda
```

El archivo .env guarda la dirección de conexión. Si incluye usuario y contraseña, debe agregarse a .gitignore y no subirse al repositorio.

Schema, Model y Document

El Schema describe los datos; el Model ofrece métodos para trabajar con la colección; cada resultado guardado es un Document.

Modelo de producto

MODELS/PRODUCTO.JS

```
const mongoose = require("mongoose");

const productoSchema = new mongoose.Schema({
  nombre: {
    type: String,
    required: true,
    trim: true
  },
  precio: {
    type: Number,
    required: true,
    min: 0
  },
  categoria: {
    type: String,
    enum: ["tecnología", "hogar", "oficina"]
  },
  disponible: {
    type: Boolean,
    default: true
  }
}, { timestamps: true });

module.exports = mongoose.model("Producto", productoSchema);
```

Reglas utilizadas

Opción	Para qué sirve	Aplicación en el ejemplo
required	Exige que el campo tenga un valor	nombre y precio
trim	Retira espacios al inicio y al final	nombre
min	Establece el valor numérico mínimo	precio desde cero
enum	Limita el campo a valores permitidos	categoría
default	Asigna un valor cuando no se envía	disponible = true
timestamps	Agrega createdAt y updatedAt	fechas automáticas

1 Schema
Define las reglas

2 Model
Producto

3 Collection
productos

4 Document
Dato guardado

CRUD con Mongoose

CRUD

Los modelos de Mongoose incluyen métodos para crear, leer, actualizar y eliminar documentos. Estas operaciones se pueden usar dentro de controladores de Express.

Métodos principales

Operación	Método de Mongoose	Resultado
Crear	Producto.create(datos)	Nuevo documento
Listar	Producto.find()	Arreglo de documentos
Consultar uno	Producto.findById(id)	Documento o null
Actualizar	Producto.findByIdAndUpdate(id, datos)	Documento actualizado
Eliminar	Producto.findByIdAndDelete(id)	Documento eliminado

Ejemplos en un controlador

LISTAR Y CREAR

```
const Producto = require("../models/Producto");

async function listar(req, res) {
  const productos = await Producto.find();
  res.json(productos);
}

async function crear(req, res) {
  const producto = await Producto.create(req.body);
  res.status(201).json(producto);
}
```

ACTUALIZAR Y ELIMINAR

```
async function actualizar(req, res) {
  const producto = await Producto.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true, runValidators: true }
  );
  res.json(producto);
}

async function eliminar(req, res) {
  await Producto.findByIdAndDelete(req.params.id);
  res.json({ mensaje: "Producto eliminado" });
}
```

MANEJO BÁSICO DE ERRORES

```
try {
  // Consulta con Mongoose
} catch (error) {
  res.status(400).json({ mensaje: error.message });
}
```

La opción `new: true` devuelve el documento ya actualizado. `runValidators: true` aplica las reglas del Schema durante la actualización.

Ejemplo dentro de Express

Organización sugerida

PROYECTO

```
tienda-api/  
|-- app.js  
|-- config/  
|   |-- database.js  
|   |-- models/  
|   |   |-- Producto.js  
|   |-- controllers/  
|   |   |-- productos.controller.js  
|   |-- routes/  
|   |   |-- productos.routes.js  
|-- .env  
|-- .gitignore
```

ROUTES/PRODUCTOS.ROUTES.JS

```
const router = require("express").Router();  
const controlador = require(  
  "../controllers/productos.controller"  
);  
  
router.get("/", controlador.listar);  
router.post("/", controlador.crear);  
router.put("/:id", controlador.actualizar);  
router.delete("/:id", controlador.eliminar);  
  
module.exports = router;
```

Archivo principal

APP.JS

```
require("dotenv").config();  
const express = require("express");  
const conectarDB = require("../config/database");  
const productosRoutes = require("../routes/productos.routes");  
  
const app = express();  
app.use(express.json());  
app.use("/api/productos", productosRoutes);  
  
conectarDB()  
  .then(() => app.listen(3000, () => {  
    console.log("Servidor en http://localhost:3000");  
  }))  
  .catch(error => console.error(error));
```

Recorrido de una petición

1 Cliente

POST /api/productos

2 Ruta

Llama al controlador

3 Mongoose

Valida y crea

4 MongoDB

Guarda el documento

5 Respuesta

201 + JSON

Método	Ruta	Acción
GET	/api/productos	Listar productos
POST	/api/productos	Crear un producto
PUT	/api/productos/:id	Actualizar un producto
DELETE	/api/productos/:id	Eliminar un producto